

RFCs

In software engineering, your programs rarely live in a vacuum. They need to talk to databases, web servers, and other apps. To do this, they must follow **Standard Protocols** and use data formats such as **JSON**.

1. What is an RFC?

An **RFC (Request for Comments)** is a formal document, primarily produced by the **Internet Engineering Task Force (IETF)**, outlining technical specifications, standards, and methodologies for Internet technologies

- **Why they matter:** They ensure that a **Go** program, a **Java** program, and a **Python** program all "speak the same language."
- **Common Examples:**
 - **HTML:** Defines how web pages are structured.
 - **HTTP (RFC 2616):** Defines how messages are sent over the web (Headers, Content, etc.).
 - **URI/URL:** Defines the format for web addresses.

2. Networking Packages in Go

Go's standard library is world-class for networking. It provides built-in packages so you don't have to write complex protocols from scratch.

- **net/http:** Used for web communication.
 - **Example:** `http.Get("http://example.com")` fetches a webpage instantly.
- **net:** A lower-level package for TCP/IP and socket programming.
 - **Example:** `net.Dial("tcp", "example.com:80")` creates a direct connection to a specific port.

3. JSON: The Universal Language of Data

JSON (JavaScript Object Notation) is the most popular format for trading data between systems. Even though it has "JavaScript" in the name, it is used by almost every programming language today. **Uses Key-value pairs**

4. Comparison: Go Struct vs. JSON

When you want to send a Go object over the network, you **"encode"** it into JSON. When you receive data, you **"decode"** it back into a Go object.

```
p1 := Person{Name: "Joe", Address: "A St", Phone: "123"}    // Go Struct

{
    "name": "Joe",
    "address": "A St",
    "phone": "123"
}                    // Equivalent JSON Object
```

Expert Takeaway:

In professional development, you will spend a huge amount of your time "**Marshalling**" (**encoding**) and "**Unmarshalling**" (**decoding**) **JSON**.

Pro Tip: The specific package you will use **99%** of the time is **encoding/json**. It allows you to take a **Struct** and turn it into a **JSON** string with a single function call (**json.Marshal**). This is how your Go backend will send data to a **React** or **Vue** frontend!